

Пользовательские Функции

Докладчик: Евграфов Михаил



Определение функций

Зачем нужны функции

- устранение дублирования
- декомпозиция
- читаемость
- повторное использование
- тестируемость
- изоляция

Проблема

```
print("=" * 20)
print("Report")
print("=" * 20)

print("=" * 20)
print("Summary")
print("=" * 20)

print("=" * 20)
print("Details")
print("=" * 20)
```



дублирование логики

Решение

```
def print_header(title: str) -> None:
    print("=" * 20)
    print(title)
    print("=" * 20)


print_header("Report")
print_header("Summary")
print_header("Details")
```

Синтаксис определения

```
def function_name(parameter_1, parameter_2):
    # тело функции - блок с отступом
    ...
```

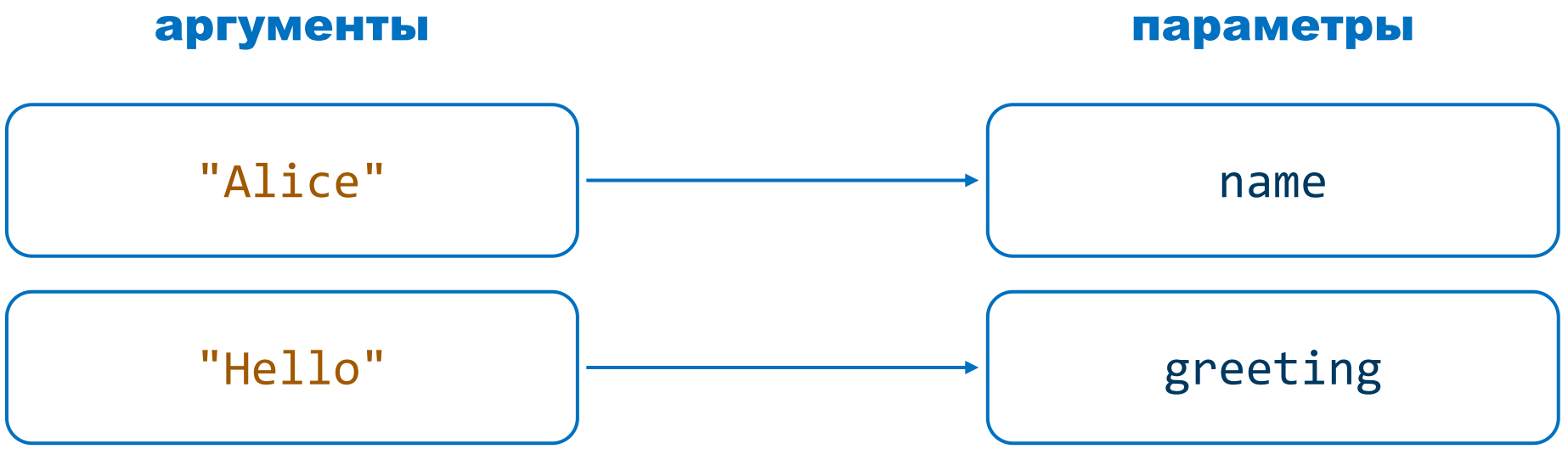
`def`	`function_name`	`(parameters)`	`:`	**тело**
ключевое слово	**идентификатор**	**список параметров**	**конец заголовка**	**блок с отступом**

Параметры функции

```
def greet(name: str, greeting: str) -> None:
    print(f"{greeting}, {name}!")


greet("Alice", "Hello")
# Hello, Alice!
greet("Bob", "Good morning")
# Good morning, Bob!
```

Параметры и аргументы



аргументы связываются с параметрами при вызове

Тело функции

```
def print_even_numbers(limit: int) -> None:
    for number in range(limit):
        if number % 2 == 0:
            print(number)
```

Пустое тело

```
def not_implemented_yet() -> None:
    pass
```

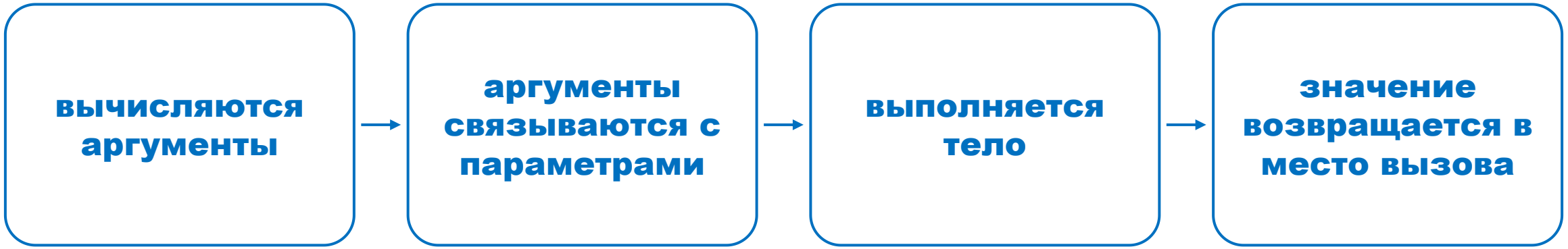
```
def not_implemented_yet() -> None:
    ...
```

Вызов и объект-функция

```
def say_hello() -> None:
    print("Hello!")

say_hello()
# Hello!
say_hello
# ничего не произошло
print(say_hello)
# <function say_hello at 0x...>
```

Порядок вызова



Время выполнения тела

```
print("before def")  # before def

def show_message() -> None:
    print("inside function")

print("after def")   # after def
show_message()       # inside function
```

def vs вызов

def	вызов
создает объект-функцию связывает объект с именем тело не выполняется	вычисляет аргументы выполняет тело возвращает значение

Следствие

```
def first() -> None:
    print("first")
    second()

def second() -> None:
    print("second")

first()
# first
# second
```

Инструкция return

```
def square(number: int) -> int:
    return number * number

result = square(4)
print(result)
# 16

print(square(3) + square(4))
# 25
```

Код после return

```
def get_number() -> int:
    return 1
    print("This line is never reached")

print(get_number())
# 1
```

Ранний возврат

```
def get_sign(number: int) -> str:
    if number > 0:
        return "positive"

    if number < 0:
        return "negative"

    return "zero"


print(get_sign(5))   # positive
print(get_sign(-3))  # negative
print(get_sign(0))   # zero
```

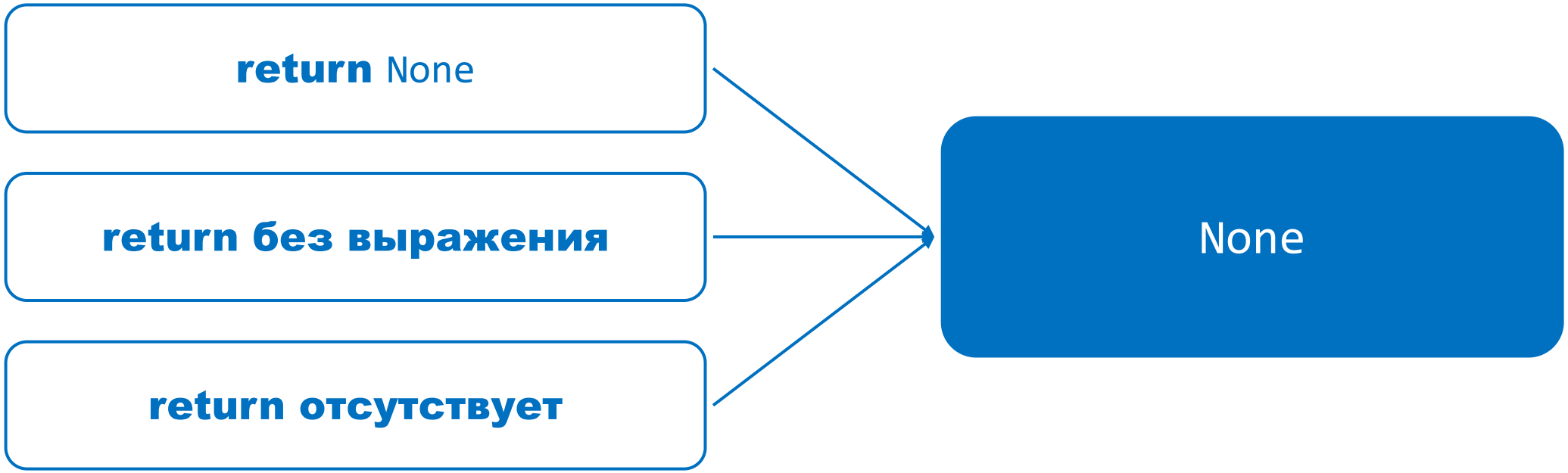
return внутри цикла

```
def find_first_even(
    numbers: list[int]
) -> int | None:
    for number in numbers:
        if number % 2 == 0:
            return number

    return None

print(find_first_even([1, 3, 4, 6]))  # 4
print(find_first_even([1, 3, 5]))     # None
```

Возврат None



Возврат None

```
def no_return() -> None:
    print("working")

def bare_return() -> None:
    print("working")
    return

first_result = no_return()      # working
second_result = bare_return()   # working
print(first_result)    # None
print(second_result)   # None
```

print vs return

```
def add_with_print(first_term: int, second_term: int) -> None:
    print(first_term + second_term)

def add_with_return(first_term: int, second_term: int) -> int:
    return first_term + second_term

result_with_print = add_with_print(2, 3)    # 5
result_with_return = add_with_return(2, 3)

print(result_with_print)          # None
print(result_with_return)         # 5
```

Возврат нескольких значений

```
def get_min_max(
    numbers: list[int]
) -> tuple[int, int]:
    return min(numbers), max(numbers)

result = get_min_max([4, 1, 7, 3])

print(result)          # (1, 7)
print(type(result))    # <class 'tuple'>
```

Распаковка результата

```
def get_min_max(
    numbers: list[int]
) -> tuple[int, int]:
    return min(numbers), max(numbers)

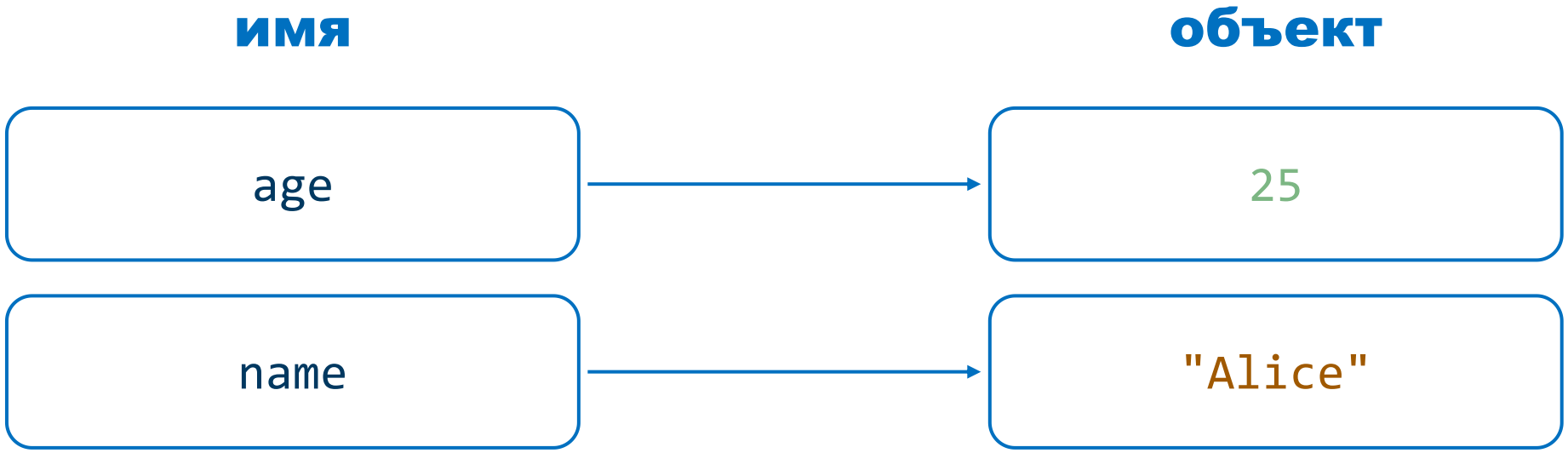
minimum, maximum = get_min_max([4, 1, 7, 3])

print(minimum)  # 1
print(maximum)  # 7
```



Пространства имен

Пространство имен



соответствие имени объекту, по сути словарь

Функция globals()

```
age = 25
name = "Alice"

print(globals()["age"])
# 25
print("name" in globals())
# True
```

Виды пространств имен

пространство	создается	удаляется
встроенное	при запуске интерпретатора	при завершении интерпретатора
глобальное	при загрузке модуля	при завершении программы
локальное	при каждом вызове функции	при завершении вызова

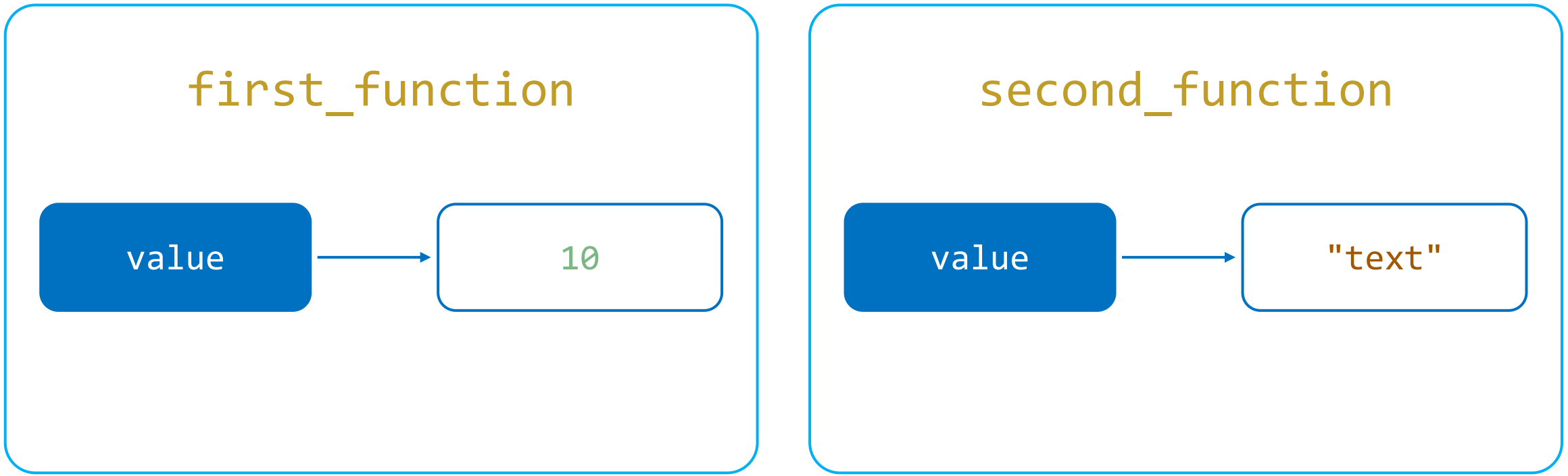
Независимость

```
def first_function() -> None:
    value = 10
    print("first:", value)

def second_function() -> None:
    value = "text"
    print("second:", value)

first_function()   # first: 10
second_function()  # second: text
```

Два пространства имен



одинаковые имена в разных пространствах не связаны

Независимость от вызовов

```
def count_to(limit: int) -> int:
    counter = 0

    for i in range(limit):
        counter += 1

    return counter


print(count_to(3))  # 3
print(count_to(5))  # 5
```

Глобальные переменные

```
tax_rate = 0.2  # глобальная переменная

def calculate_price(price: float) -> float:
    return price * (1 + tax_rate)

print(calculate_price(100))
# 120.0
```

Локальные переменные

```
def calculate_total(
    price: float,
    count: int,
) -> float:
    total = price * count  # локальная переменная
    return total

print(calculate_total(10, 3))
# 30
print(total)
# NameError: name 'total' is not defined
print(price)
# NameError: name 'price' is not defined
```

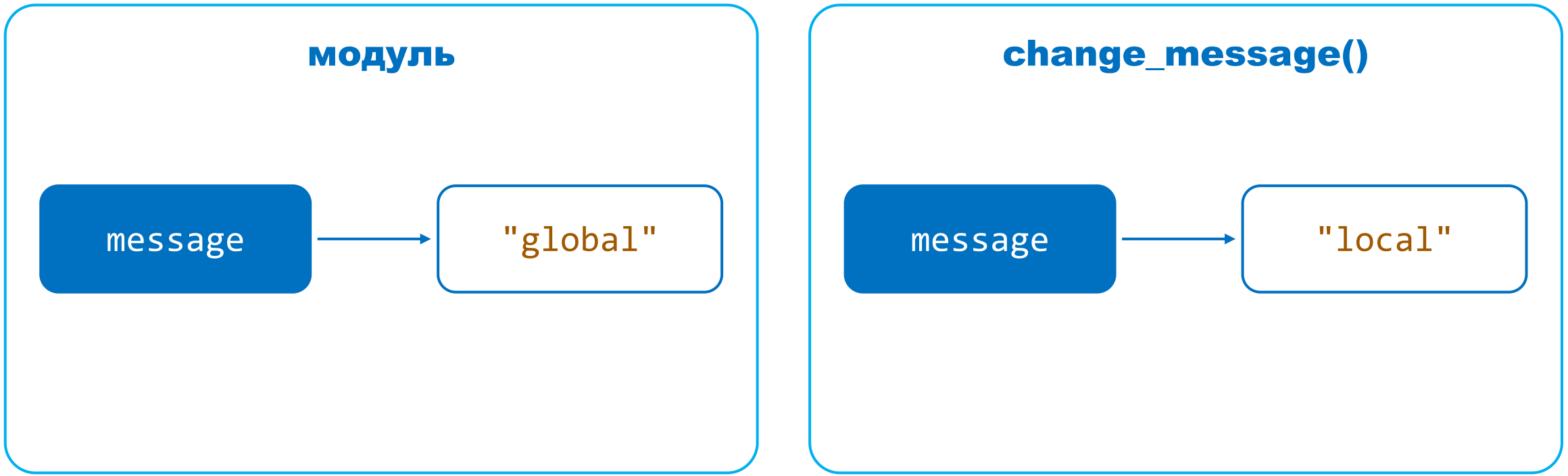
Затенение

```
message = "global"

def change_message() -> None:
    message = "local"
    print("inside:", message)

change_message()                # inside: local
print("outside:", message)      # outside: global
```

Затенение

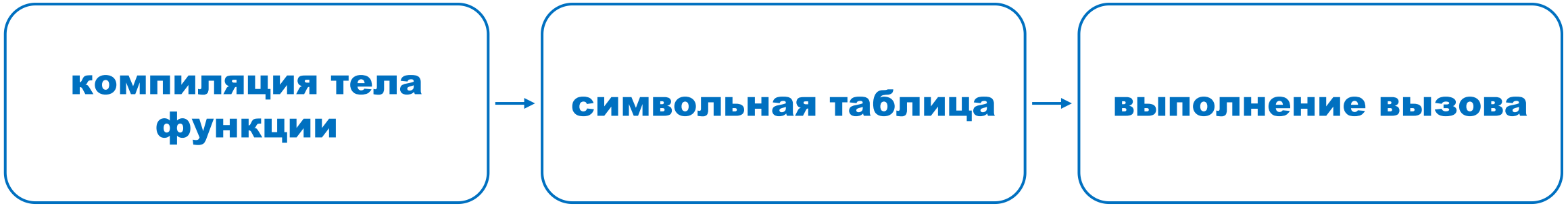


локальное имя закрывает глобальное на время вызова



Поиск имен

Построение таблицы имен



Классификация имен

локальные

параметры
имена с присваиванием в теле
=, +=, for, import, def, with

внешние

имена только читаются
ищутся во внешних областях

Символьная таблица

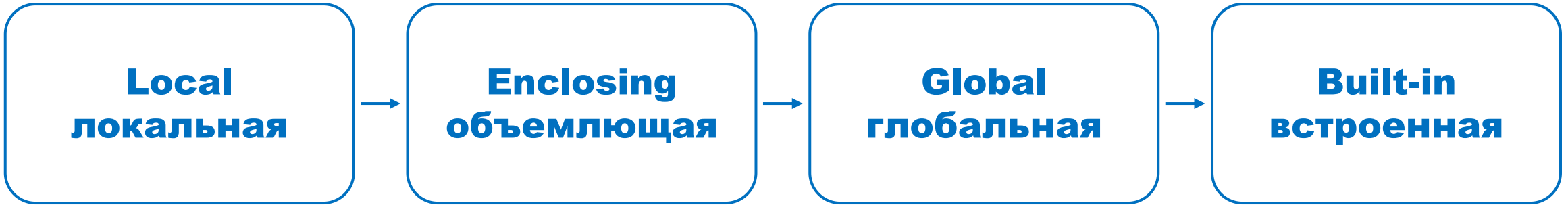
```
rate = 2

def calculate(price: float, count: int) -> float:
    total = price * count
    return total * rate

code = calculate.__code__

print(code.co_varnames)
# ('price', 'count', 'total')
print(code.co_names)
# ('rate',)
```

Правило LEGB



Поиск имен

```
var_global = 42

def nesting_function() -> None:
    var_nonlocal = 142
    def nested_function() -> None:
        var_local = 242
        print(var_local)
        print(var_nonlocal)
        print(var_global)
    nested_function()

nesting_function()
# 242
# 142
# 42
```



Чтение vs присваивание

чтение имени	присваивание
поиск: L, затем E, G, B побеждает ближайшее не найдено: NameError	имя всегда локальное создается в текущей функции



Передача аргументов

Передача ссылки

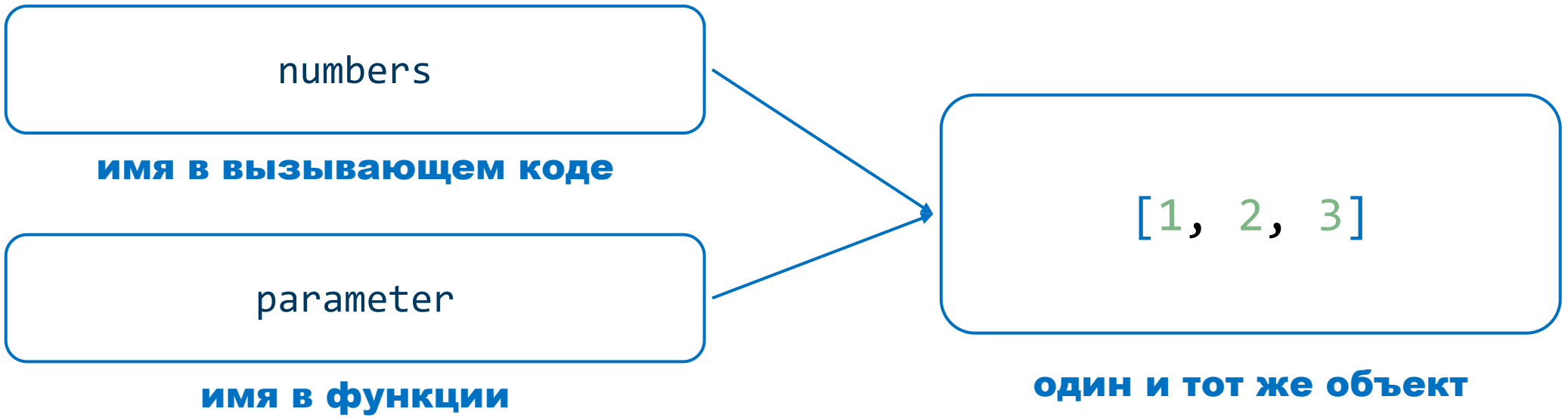
```
def show_identity(parameter: list[int]) -> None:
    print("inside:", id(parameter))

numbers = [1, 2, 3]

print("outside:", id(numbers))
# outside: 140234567890123

show_identity(numbers)
# inside: 140234567890123
```

Параметр — новое имя



Модели передачи

по значению	по ссылке на переменную	**по ссылке на объект**
копия объекта, в Python нет	как & в C++, в Python нет	модель Python

Изменяемые аргументы

```
def add_item(
    items: list[str],
    item: str,
) -> None:
    items.append(item)

shopping_list = ["bread", "milk"]
add_item(shopping_list, "eggs")

print(shopping_list)
# ['bread', 'milk', 'eggs']
```

Присваивание в теле

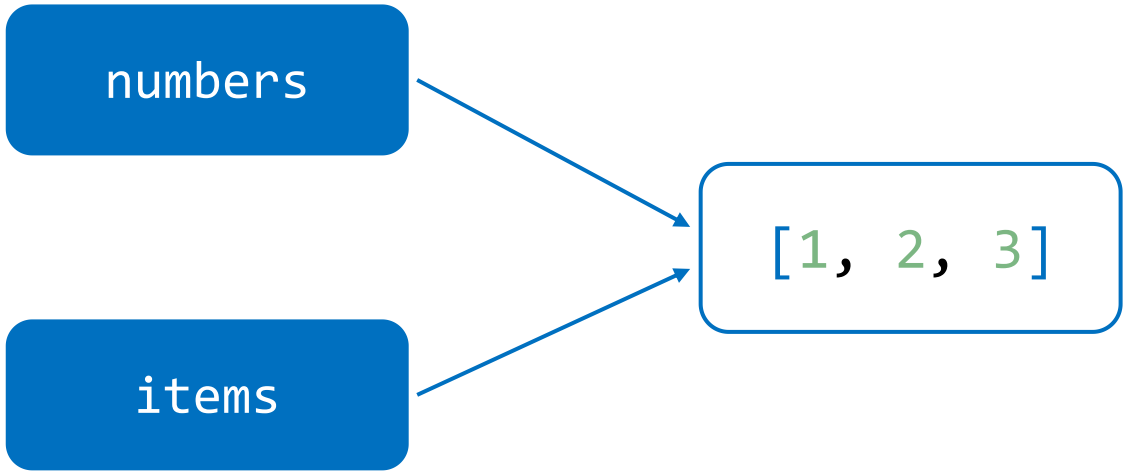
```
def replace_list(items: list[int]) -> None:
    items = [100, 200]
    print("inside:", items)

numbers = [1, 2, 3]

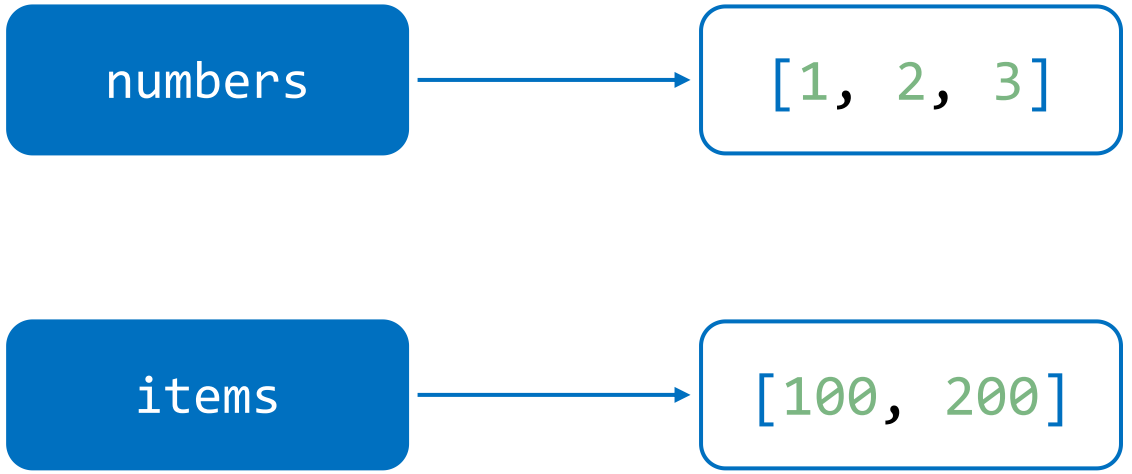
replace_list(numbers)
# inside: [100, 200]
print("outside:", numbers)
# outside: [1, 2, 3]
```

Присваивание в теле

до присваивания



после присваивания



присваивание меняет только локальное имя

Способы передачи

по позиции

соответствие по порядку
первый аргумент — первому параметру

по имени

соответствие по имени
порядок не важен

Способы передачи

```
def describe_user(name: str, age: int, city: str) -> str:
    return f"{name}, {age}, {city}"

# по позиции
print(describe_user("Alice", 25, "Paris"))
# по имени
print(describe_user(city="Paris", name="Alice", age=25))
# смешанный подход
print(describe_user("Alice", city="Paris", age=25))
print(describe_user(name="Alice", 25, "Paris"))
# SyntaxError: positional argument follows keyword argument
```

Ошибки передачи

```
describe_user("Alice", 25)
# TypeError: missing 1 required positional argument: 'city'

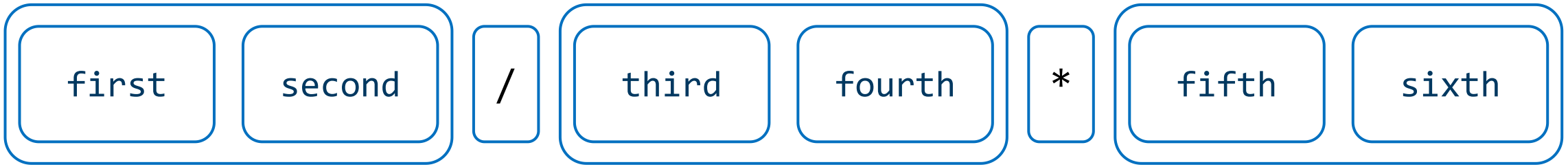
describe_user("Alice", 25, "Paris", "extra")
# TypeError: takes 3 positional arguments but 4 were given

describe_user("Alice", 25, "Paris", country="FR")
# TypeError: got an unexpected keyword argument 'country'

describe_user("Alice", name="Bob", age=25, city="")
# TypeError: got multiple values for argument 'name'
```

Схема параметров

```
def function(first, second, /, third, fourth, *, fifth, sixth):
```



только по позиции

по позиции или по имени

только по имени

Виды параметров

параметры	расположение	передача
строго позиционные	до /	только по позиции
обычные	между / и *	по позиции или по имени
строго именованные	после *	только по имени

Строго позиционные

```
def power(base: float, exponent: float, /) -> float:
    return base ** exponent

print(power(2, 10))
# 1024

print(power(base=2, exponent=10))
# TypeError: ...
```

Строго именованные

```
def create_window(
    width: int,
    height: int,
    *,
    resizable: bool,
) -> str:
    return f"{width}x{height}, resizable={resizable}"

print(create_window(800, 600, resizable=True))
# 800x600, resizable=True
print(create_window(800, 600, resizable=False))
# 800x600, resizable=False
print(create_window(800, 600, True))
# TypeError: takes 2 positional arguments but 3 were given
```

Порядок именованных

```
def connect(host: str, *, port: int, user: str) -> str:
    return f"{user}@{host}:{port}"

print(connect("localhost", port=5432, user="admin"))
# admin@localhost:5432

print(connect("localhost", user="admin", port=5432))
# admin@localhost:5432
```

Оба ограничения

```
def build_url(
    scheme: str,
    host: str,
    /,
    port: int,
    *,
    path: str,
    secure: bool,
) -> str:
    protocol = f"{scheme}{'s' if secure else ''}"
    return f"{protocol}://{host}:{port}{path}"

print(build_url("http", "example.com", 80, path="/", secure=False))
# http://example.com:80/
```

Параметр *args

```
def show_arguments(*args: int) -> None:
    print(args)
    print(type(args))

show_arguments(1, 2, 3)
# (1, 2, 3)
# <class 'tuple'>
```

Параметр *args

```
def sum_all(*args: float) -> float:
    total = 0

    for number in args:
        total += number

    return total

print(sum_all())                # 0
print(sum_all(1))               # 1
print(sum_all(1, 2, 3, 4, 5))   # 15
```

Один параметр со *

```
def invalid_function(
    *first_args: int,
    *second_args: int,
) -> None:
    pass

# SyntaxError: * argument may appear only once
```

Параметр **kwargs

```
def show_options(**kwargs: int) -> None:
    print(kwargs)
    print(type(kwargs))

show_options(width=800, height=600)
# {'width': 800, 'height': 600}
# <class 'dict'>
```

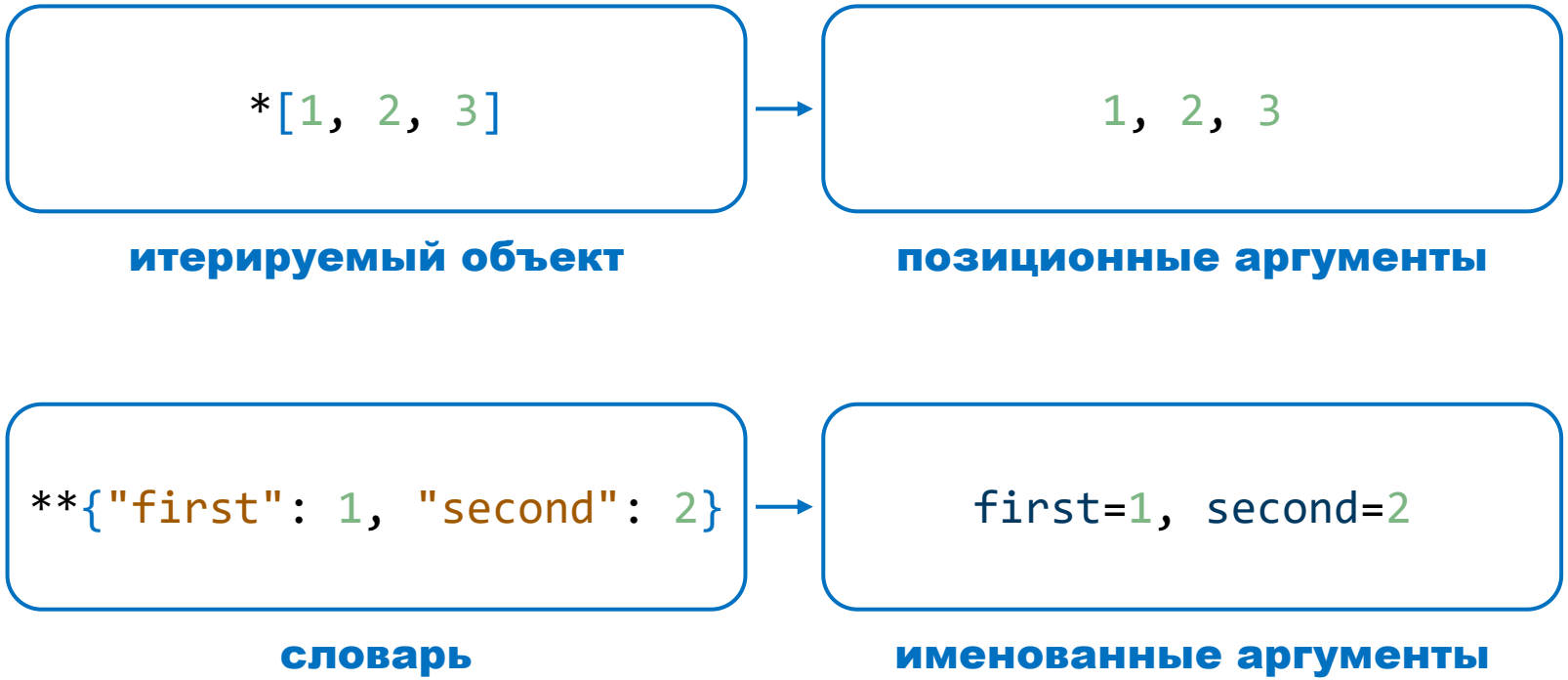
Порядок параметров



Все виды параметров

```
def function(
    positional_only: int,
    /,
    normal: int,
    *args: int,
    keyword_only: int,
    **kwargs: int,
) -> None:
    ...
```

Распаковка при вызове



Распаковка *

```
def add_three(first: int, second: int, third: int) -> int:
    return first + second + third

numbers = [1, 2, 3]
print(add_three(*numbers))
# 6 - то же, что add_three(1, 2, 3)
print(add_three(0, *[5, 6]))
# 11
print(add_three(*range(3)))
# 3
```

Распаковка **

```
def add_three(first: int, second: int, third: int) -> int:
    return first + second + third


values = {"first": 1, "second": 2, "third": 3}
print(add_three(**values))
# 6 - то же, что add_three(first=1, second=2, third=3)
print(add_three(1, **{"second": 5, "third": 6}))
# 12
```



Значения по умолчанию

Значение по умолчанию

```
def greet(
    name: str,
    greeting: str = "Hello",
    punctuation: str = "!",
) -> str:
    return f"{greeting}, {name}{punctuation}"

print(greet("Alice"))                    # Hello, Alice!
print(greet("Alice", "Hi"))              # Hi, Alice!
print(greet("Alice", punctuation="?"))   # Hello, Alice?
print(greet("Alice", "Good morning", "."))
# Good morning, Alice.
```

Ограничения

```
def bad(first: int = 1, second: int) -> None:
    pass

# SyntaxError
```

Время вычисления

```
def get_default() -> int:
    print("evaluating default")
    return 10

print("before def")
# before def

def show(value: int = get_default()) -> None:
    print("value:", value)
# evaluating default

print("after def")
# after def
```

Атрибут __defaults__

```
def get_default() -> int:
    print("evaluating default")
    return 10

def show(value: int = get_default()) -> None:
    print("value:", value)
# evaluating default

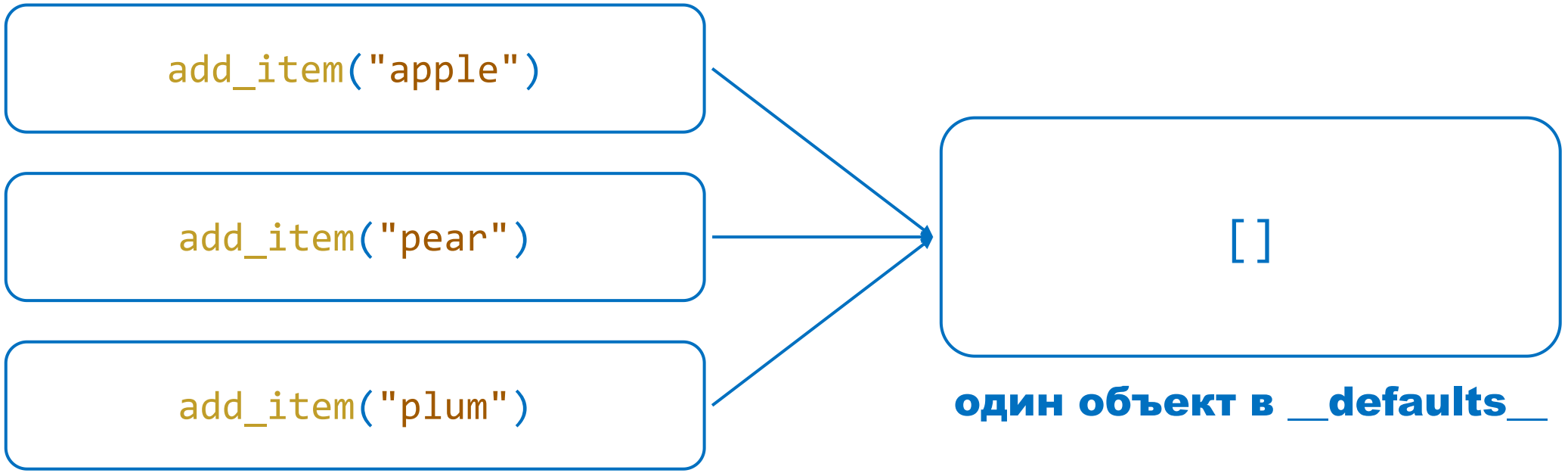
print(show.__defaults__)
# (10,)
```

Проблема

```
def add_item(
    item: str,
    items: list[str] = [],
) -> list[str]:
    items.append(item)
    return items

print(add_item("apple"))  # ['apple']
print(add_item("pear"))   # ['apple', 'pear']
print(add_item("plum"))   # ['apple', 'pear', 'plum']
```

Причина


add_item("apple")
add_item("pear")
add_item("plum")
[]
один объект в __defaults__

Причина

```
first_result = add_item("a")
second_result = add_item("b")

print(first_result is second_result)
# True

print(add_item.__defaults__)
# (['a', 'b'],)
```

Решение 1: None

```
def add_item(
    item: str, items: list[str] | None = None
) -> list[str]:
    if items is None:
        items = []

    items.append(item)
    return items

print(add_item("apple"))  # ['apple']
print(add_item("pear"))   # ['pear']
print(add_item("plum"))   # ['plum']
```

Ограничения

```
def get_value(
    data: dict[str, int | None],
    key: str,
    default: int | None = None,
) -> int | None:
    if key in data:
        return data[key]

    if default is None:
        raise KeyError(key)

    return default

user_data = {"age": 30}
print(get_value(user_data, "city"))         # KeyError: 'city'
print(get_value(user_data, "city", None))  # KeyError: 'city'
```

Решение 2: sentinel

```
sentinel = object()

def get_value(
    data: dict[str, int | None],
    key: str,
    default: object = sentinel,
) -> object:
    if key in data:
        return data[key]

    if default is sentinel:
        raise KeyError(key)

    return default

user_data = {"age": None}
print(get_value(user_data, "city", None))  # None
print(get_value(user_data, "city"))        # KeyError: 'city'
```



Метаинформация

docstring

```
def square(number: int) -> int:
    """Return the square of a number."""
    return number * number

print(square.__doc__)
# Return the square of a number.
```

docstring

```
def square(number: int) -> int:
    """Return the square of a number."""
    return number * number

help(square)
# Help on function square in module __main__:
#
# square(number: int) -> int
#     Return the square of a number.
```

Комментарий vs docstring

комментарий	docstring
игнорируется интерпретатором	хранится в __doc__
читает исходный код	использует функцию
почему и как реализовано	что делает функция

Многострочный docstring

```
def divide(dividend: float, divisor: float) -> float:
    """Divide one number by another.

    Args:
        dividend: The number to be divided.
        divisor: The number to divide by.

    Returns:
        The result of the division.

    Raises:
        ZeroDivisionError: If divisor is zero.
    """
    return dividend / divisor
```

Аннотации типов

```
def repeat_text(text: str, count: int = 2) -> str:
```

```
text: str
```

тип параметра

```
count: int = 2
```

тип и значение по умолчанию

```
-> str
```

тип результата

Аннотация -> None

```
def print_header(title: str) -> None:
    print("=" * 20)
    print(title)
```

Аннотации *args и **kwargs

```python
def sum_all(*args: int) -> int:
    return sum(args)
# args имеет тип tuple[int, ...]


def build_tag(tag_name: str, **attributes: str) -> str:
    ...
# attributes - dict[str, str]
```

Игнорирование аннотаций

```
def add(first_term: int, second_term: int) -> int:
    return first_term + second_term

print(add(2, 3))       # 5
print(add("a", "b"))   # ab
print(add(1.5, 2))     # 3.5

print(add("a", 1))
# TypeError: ...
```

Проверка аннотаций

интерпретатор

не проверяет типы
хранит аннотации в
__annotations__

инструменты

mypy, pyright
редакторы и IDE
генераторы документации
pydantic, FastAPI

Аннотации коллекций

```
def get_average(numbers: list[int]) -> float:
    ...

def count_words(words: list[str]) -> dict[str, int]:
    ...
```

Аннотации коллекций

```
def get_point() -> tuple[int, int]:
    ...


def get_scores() -> tuple[int, ...]:
    ...


def get_unique(items: list[str]) -> set[str]:
    ...
```

Значение или None

```
from typing import Optional


def find_index(items: list[str], target: str) -> Optional[int]:
    ...


def find_index(items: list[str], target: str) -> int | None:
    ...
```

Объединение

```
from typing import Union


def to_number(value: int | str) -> int:
    ...


def to_number_old(value: Union[int, str]) -> int:
    ...
```

Тип Any

```
from typing import Any


def describe(value: Any) -> str:
    ...
```

Callable

```
from typing import Callable


def apply_twice(
    function: Callable[[int], int],
    value: int,
) -> int:
    ...
```

Literal

```
from typing import Literal

def open_file(mode: Literal["r", "w", "a"]) -> None:
    ...
```



Функции как объекты

Тип функции

```
def square(number: int) -> int:
    return number * number


print(type(square))
# <class 'function'>
```

Атрибуты функции

```
print(square.__name__)
# square - имя функции
print(square.__doc__)
# None - docstring
print(square.__defaults__)
# None - значения по умолчанию
print(square.__annotations__)
# {'number': <class 'int'>, 'return': <class 'int'>}
```

Функции и переменные

```
def square(number: int) -> int:
    return number * number

calculate = square

print(calculate(4))           # 16
print(square(4))              # 16
print(calculate is square)    # True
```

Функции в коллекциях

```
def square(number: int) -> int:
    return number * number


def cube(number: int) -> int:
    return number * number * number

functions = [square, cube, abs]

for function in functions:
    print(function.__name__, function(-3))
```

Функция как аргумент

```
from typing import Callable

def apply_to_all(
    function: Callable[[int], int],
    numbers: list[int],
) -> list[int]:
    return [
        function(number) for number in numbers
    ]

def square(number: int) -> int:
    return number * number

print(apply_to_all(square, [1, 2, 3]))  # [1, 4, 9]
print(apply_to_all(abs, [-1, -2, 3]))   # [1, 2, 3]
```

Возврат функции

```
from typing import Callable

def make_greeter() -> Callable[[str], str]:
    def greet(name: str) -> str:
        return f"Hello, {name}!"

    return greet

greeter = make_greeter()
print(greeter("Alice"))
# Hello, Alice!
```



Спасибо за внимание!